Universidade Federal do Rio Grande do Norte — Departamento de Informática e Matemática Aplicada DIM0548 – Engenharia de Linguagens

Subproblema 7 — Análise Sintática: Documentação do Analisador Sintático

Grupo: FLAWBERT LORRAN DA SILVA COSTA, RODRIGO FERREIRA DA NOBREGA, CAIO DANIEL FONSECA DE ARAÚJO, MURILO ANTONIO LIMA DA COSTA

Referências: SEBESTA, R. *Concepts of Programming Languages*, 11ª ed., Addison-Wesley, 2016. APPEL, A. *Modern Compiler Implementation in C*, Cambridge University Press, 1998 (cap. 3). Repositório: `https://github.com/Caio2a7/el-lang`

1. Estrutura dos programas da linguagem proposta

Um programa é uma sequência de subprogramas (`proc`/`func`), opcionalmente precedida de constantes nomeadas — gramática BNF definida no Problema 5, item 4.2.

1.1 Código-fonte mínimo compilável, um procedimento sem parâmetros e corpo vazio:

```
proc main() {
}
```

A gramática não exige que o procedimento se chame `main` — isso é convenção de uso, verificável só na análise semântica.

1.2 Exemplo geral, exercitando constante, array, função, `if/else-if/else`, `while` e `for`:

```
const int LIMITE := 500;
func taxa(int valor, int perc) -> int { return (valor * perc) / 100; }
proc processa(int ordem, out int status) {
    int cobranca := taxa(ordem, 2);
    int[LIMITE] hist := [0; LIMITE];
    if (cobranca > LIMITE) { status := 0; }
    else if (cobranca == 0) { status := 1; }
    else { registra(ordem); status := 2; }
    while (cobranca > 0) { cobranca := cobranca - 1; }
    for (int i := 0 to 9) { hist[i] := i; }
}
```

Dois programas completos estão entre os arquivos de teste: `tests/valido/02_mergesort.txt` (Mergesort do item 4.1 do Problema 5) e `tests/valido/03_funcao_juros.txt` (função de juros do item 6.c do Problema 4).

2. Estrutura dos analisadores

2.1 Analisador Léxico

Tarefas: converte a cadeia de caracteres em tokens (identificadores, literais inteiros, 19 palavras reservadas, operadores e delimitadores do item 1.2 do Problema 5); descarta espaços e comentários `//`; mantém `yylineno`

para erros.

Decisões: palavras reservadas são listadas antes da regra genérica de identificador — o Flex escolhe o casamento mais longo e, em empate, a primeira regra declarada, garantindo que `if`, `while` etc. nunca virem identificadores. Prefixos compartilhados (`</<=`, `=/:=`) não exigem tratamento especial, pois o casamento mais longo já resolve.

Dificuldades: o léxico reserva `switch`, `case`, `default` e `.` (item 1.2 do Problema 5), mas a gramática do item 4.2 não os usa em nenhuma produção — decisão já registrada pelo grupo. O lexer continua reconhecendo esses tokens; quem os rejeita é o parser. Identificadores/literais são só copiados como string (`strdup`); tabela de símbolos fica para a análise semântica.

2.2 Analisador Sintático

Tarefas: o parser (Bison, modo Yacc) decide, via autômato LALR(1) *shift-reduce*, se a sequência de tokens pertence à linguagem; em erro, informa linha e lexema e para no primeiro símbolo inesperado.

Decisões: a gramática do item 4.2 já foi desenhada (Problema 5, itens 2.2/3.4) para ser não ambígua: operadores em níveis de não-terminais (`expr→conj→rel→soma→termo→fator`) com recursão à esquerda fixam precedência e associatividade sem `%left`/`%right`. Chaves obrigatórias em `if/while/for` eliminam o *dangling-else* por construção. Optou-se por LALR (não LR(1) canônico) porque funde estados de mesmo núcleo LR(0), reduzindo a tabela para 157 estados sem perder poder prático para esta gramática (Appel, cap. 3).

Conflitos: nenhum shift/reduce ou reduce/reduce foi reportado (`y.output`, 1911 linhas, zero ocorrências de "conflict"). Casos que poderiam gerar ambiguidade foram checados: `<alvo>` (`id` ou `id[expr]`) e `<chamada>` (`id(...)`) dividem o prefixo `id`, mas o token seguinte decide com 1 símbolo de lookahead; `<decl_var>` começa com palavra-chave de tipo, nunca com `id`, não conflitando com `<atrib>`/`<chamada>`; o `-` unário (`<fator>`) e o `-` binário (`<soma>`) ocupam posições distintas da gramática. O único aviso do Bison foi "terminais não usados" para `SWITCH`, `CASE`, `DEFAULT`, `DOT` — consequência esperada do item 2.1.

2.3 Limitações da implementação atual

- Reconhecedor sintático puro: aceita/rejeita e localiza o erro, mas não constrói AST nem faz análise semântica (tipos, declaração prévia, posse de referências — Problema 5, itens 5.1/5.3).
- `switch/case/default` e `.` são léxicos válidos mas sintaticamente rejeitados (sem produção).
- Recuperação de erro padrão do Yacc: para no primeiro símbolo inesperado.
- Sem tabela de símbolos nem liberação de memória (`strdup` apenas).
- A gramática não exige, por si, um procedimento `main`.

3. Uso do analisador sintático

3.1 Como gerar

Pré-requisitos: `flex`, `bison`, `gcc`.

```
git clone https://github.com/Caio2a7/el-lang.git
cd el-lang
make            # bison -d -v (y.tab.c/.h, y.output) + flex (lex.yy.c) + gcc -> compilador
```

```
make test       # roda tests/valido e tests/invalido
make conflitos  # confirma ausência de conflitos em y.output
```

O executável `compilador` lê o programa pela entrada padrão.

3.2 Programas de teste e resultados esperados

Arquivo (válido)	O que testa	Resultado
`01_minimo.txt`	Programa mínimo	`ACEITO`
`02_mergesort.txt`	`const`, arrays, `out`, recursão, `while`, `for`	`ACEITO`
`03_funcao_juros.txt`	Função com retorno em expressão	`ACEITO`
`04_else_if.txt`	Cadeia `if/else-if/else`, chamada como comando	`ACEITO`

Arquivo (inválido)	Erro esperado	Resultado
`01_erro_lexico.txt`	Caractere fora do alfabeto (`@`)	erro léxico + sintático, `REJEITADO`
`02_sem_chaves.txt`	`if` sem chaves obrigatórias	`linha 2: syntax error perto de 'debita'`, `REJEITADO`
`03_sem_inicializacao.txt`	Declaração sem `:=`	`linha 2: syntax error perto de ';'`, `REJEITADO`
`04_atrib_com_igual.txt`	`=` no lugar de `:=`	`linha 2: syntax error perto de '='`, `REJEITADO`

Em todos os 8 casos a saída obtida coincidiu com a esperada. Em `01_erro_lexico.txt` o léxico sinaliza o caractere inválido e descarta o símbolo; o fluxo de tokens resultante fica incompleto, e o parser também relata erro sintático na sequência — consistente com a separação léxico/sintático do Problema 5 (item 1.3).